

Eventos internos de Spring

El warm-up de caché — parte 1 de 2

Qué vas a construir hoy

Ya sabes que tu aplicación es multihilo (pool de Tomcat, listeners de RabbitMQ). Hoy construyes una pieza nueva que se apoya en eso: el warm-up de una caché, repartido en dos apartados.

Hoy montas la caché y el evento que avisa cuando deja de ser válida. En el próximo apartado, el listener que reacciona a ese aviso.

Qué es una caché

Una caché guarda el resultado de una operación cara para reutilizarlo sin repetir el trabajo.

Momento	Sin caché	Con caché
1ª petición	Paga el coste completo	Paga el coste completo, y lo guarda
2ª petición (nada cambió)	Vuelve a pagar el coste completo	Instantánea — usa lo ya guardado
Tras crear/editar/borrar un libro	Paga el coste completo (como siempre)	Paga el coste completo otra vez — la caché ya no vale

Invalidar borra el resultado guardado. Recalentar lo vuelve a calcular por adelantado, antes de que alguien lo pida.

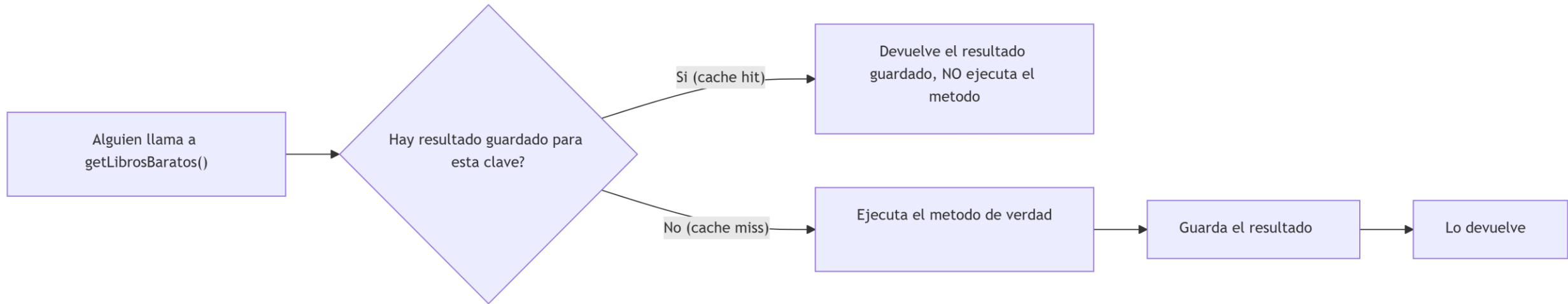
Cómo funciona @Cacheable por debajo

@Cacheable("librosBaratos") le dice a Spring que, antes de ejecutar el método de verdad, siga estos dos pasos:

- Calcula la clave que corresponde a esta llamada, y comprueba si ya hay algo guardado bajo esa clave.
- Si lo hay, lo devuelve directamente — sin ejecutar ni una línea del método. Si no lo hay, ejecuta el método, guarda lo que devuelva, y a partir de ahora responde con eso.

Sin @EnableCaching en la clase principal, @Cacheable/@CacheEvict se quedan ahí escritas pero Spring las ignora por completo, en silencio.

El flujo de una llamada cacheada



"librosBaratos" es el nombre de la caché completa, no una clave dentro de ella — la clave se construye a partir de los argumentos de la llamada.

@EnableCaching puede romper tests que ya tenías

Atención

@EnableCaching se activa para TODA la aplicación, no solo para el método que te interesa cachear. Si ya tenías tests @WebMvcTest, cargan tu clase principal (y con ella @EnableCaching), pero un test de solo el nivel web no trae la autoconfiguración que da soporte real a la caché — Spring no encuentra dónde guardar nada, y el test falla al arrancar el contexto, no al ejecutar ninguna aserción.

Es un buen ejemplo de por qué un test "de una sola capa" sigue arrastrando piezas de configuración global de tu aplicación, aunque no las necesite.

La otra mitad: @CacheEvict invalida

	@Cacheable	@CacheEvict
Va sobre	Un método de lectura	Un método de escritura (create/update/delete)
Qué hace	Guarda el resultado la primera vez; lo reutiliza después	Borra el resultado guardado
Con allEntries = true	No aplica	Borra todas las entradas de esa caché de golpe

Interesa allEntries = true porque cualquier escritura sobre el catálogo puede cambiar cuál es la lista de "más baratos" — no vale la pena intentar borrar solo la entrada afectada.

Qué es Redis

Redis es una base de datos en memoria, de tipo clave-valor, muy rápida, usada típicamente como caché compartida.

	Caché en memoria de la app	Caché en Redis
¿Sobrevive a un reinicio?	No — se pierde entera	Sí — vive en su propio contenedor
¿Se comparte entre instancias?	No — cada instancia tiene la suya	Sí — todas leen del mismo Redis
¿Dónde vive el dato?	En un mapa dentro de la JVM	En el contenedor Redis, aparte

Redis exige que lo que guardes sea Serializable

Atención

Para enviar un objeto por la red hasta Redis, Spring primero tiene que serializarlo. Por defecto, RedisCacheManager usa la serialización estándar de Java, que exige que la clase implemente Serializable. Un record como el que devuelve `getLibrosBaratos()` no la implementa por defecto, así que guardarlo en caché tal cual falla con un error de serialización.

La solución más simple y fiable: que el propio DTO implemente Serializable — un record puede hacerlo sin cambiar nada más de su definición.

Por qué esto todavía no basta

	Invalidar (@CacheEvict)	Recalentar (volver a llamar)
¿Dónde ocurre?	Dentro de create()/update()/delete(), en el hilo de la petición	Ahí mismo, si lo llamaras a continuación
¿Cuánto cuesta?	Poco — borrar una entrada de Redis	Mucho — la misma consulta lenta
¿Quién paga ese coste?	—	El cliente que solo quería crear un libro

La misma solución de siempre: que create()/update()/delete() se limiten a anunciar que la caché se ha invalidado, sin saber quién, cómo o cuándo se encarga de recalentarla. Eso es un evento.

Qué es un evento (observador / publicador-suscriptor)

Un evento es "algo ha pasado", empaquetado como un objeto. Dos papeles: quien publica (anuncia, sin saber quién escucha) y quien escucha (reacciona, sin que el publicador lo conozca).

Publicar una foto en redes sociales: no sabes ni te importa cuántas apps reaccionarán (notificar seguidores, generar miniatura, actualizar contador) — cada una se suscribe por su cuenta.

Ya has usado este patrón: los eventos de una interfaz gráfica, y RabbitMQ (Actividad 3.1) — un broker externo con productores y consumidores que no se conocen. Mismo patrón, distinta escala.

ApplicationEventPublisher: eventos dentro de la JVM

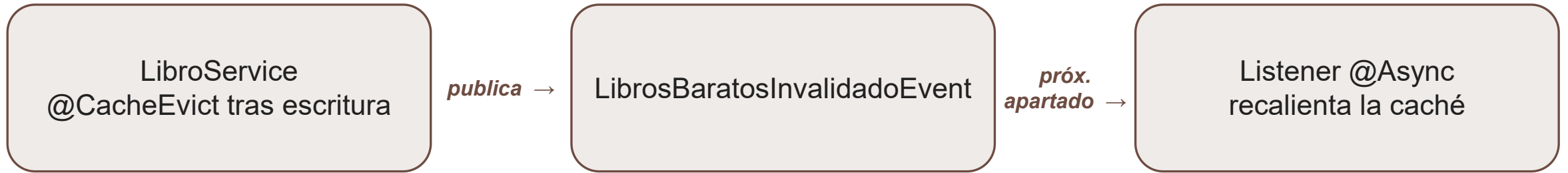
Para "la caché se ha invalidado" no hace falta nada tan pesado como un broker externo: Spring ofrece una versión más ligera del mismo patrón, dentro de la propia aplicación.

No lo confundas con RabbitMQ

RabbitMQ: mensajería entre procesos/módulos, a través de un broker externo.

ApplicationEventPublisher: eventos dentro de la misma JVM, entre beans de la misma aplicación, sin ningún broker — mucho más ligero, pero solo sirve dentro del propio proceso.

El diseño del warm-up (pieza 1 de 2)



Cuando LibroService invalida la caché (en create/update/delete, cada uno con @CacheEvict), publica además un evento interno. Sin listener todavía, publicarlo no hace nada visible — el sistema queda "emitiendo" a la espera de la pieza 2.

La clase de evento: un record inmutable

```
public record LibrosBaratosInvalidadoEvent(Instant momento) {}
```

Un record es inmutable por diseño, y eso importa cuando el objeto viaja entre hilos: no puede cambiar después de crearse, así que no hay riesgo de que un hilo lo modifique mientras otro lo lee — sin `synchronized` ni `locks`.

La publicación

```
@CacheEvict(value = "librosBaratos", allEntries = true,
            beforeInvocation = true)
@Transactional
public LibroResponseDTO create(LibroCreatedDTO dto) {
    // ... lógica de creación ya existente ...
    eventPublisher.publishEvent(
        new LibrosBaratosInvalidadoEvent(Instant.now()));
    return mapToDTO(saved);
}
```

La misma línea va también en `update()` y `delete()`, junto a su propio `@CacheEvict`: un libro editado o borrado puede cambiar igual de bien cuáles son los "más baratos".

beforeInvocation = true no es un detalle menor

Atención

Por defecto, `@CacheEvict` elimina la entrada después de que el método termine correctamente. Con `beforeInvocation = true` lo hace ANTES de entrar en `create()/update()/delete()`, garantizando que la entrada anterior ya se ha eliminado cuando se publica el evento.

La contrapartida: si la escritura termina fallando, la caché ya habrá sido vaciada y tendrá que calcularse de nuevo más adelante — una decisión sencilla y adecuada aquí, no una solución general a todo problema de concurrencia.

El mapa completo

Pieza	¿Dónde se construye?	¿Ya tiene efecto visible?
Caché con Redis (@Cacheable/@CacheEvict)	Este apartado	Sí — la 2ª petición ya sale gratis
Evento + publicación	Este apartado	No — nadie lo escucha todavía
Listener @Async que recalienta	Próximo apartado	—

Sigue pendiente el "primer usuario tras una escritura": paga el coste completo, porque la caché se acaba de invalidar y nadie la ha vuelto a llenar. Ese alivio llega en el próximo apartado.

Ideas clave

- Una caché guarda el resultado de una operación cara. Invalidar borra un resultado que ya no es válido; recalentar lo vuelve a calcular por adelantado.
- `@Cacheable` intercepta la llamada: si hay resultado guardado lo devuelve sin ejecutar el método; si no, lo ejecuta y lo guarda. `@CacheEvict(allEntries = true)` borra todo — con `beforeInvocation = true`, antes de ejecutar el método.
- `@EnableCaching` activa el mecanismo para toda la aplicación, y puede romper tests de una sola capa que carguen la clase principal.
- Redis: base de datos en memoria clave-valor, como caché compartida real — sobrevive a un reinicio. Por defecto exige `Serializable`.
- Invalidar (barato) y recalentar (caro) hay que separarlos, para que el coste de recalentar no se lo lleve por delante quien hizo la escritura.
- Un evento es "algo ha pasado"; publicador y suscriptor no se conocen entre sí. `ApplicationEventPublisher`: eventos dentro de la JVM, distinto de RabbitMQ (broker externo).
- Un evento como record es inmutable, seguro de compartir entre hilos sin locks.